COMPANION STUDY · PREPRINT

Iteration Drift and the Lattice Landscape: A Companion to the GLM

Does the Universal Binary Principle's no-floats rule actually matter? And what is the landscape of code-lattice pairings that the GLM sits inside?

Key result (iteration test). The exact (Fraction) method has **zero drift** — it is the GLM substrate and the reference. IEEE-754 with lossless handoff has relative error **~10^{-14}** (just accumulated IEEE-754 rounding). IEEE-754 with display6 handoffs (the LLM tool-output regime) has relative error **~10^{-5}** at $p=3$ accumulative — the float loses ~5 significant digits of a value that reaches $|X_{200}| \approx 6.5 \times 10^{24}$.

Lattice survey (run live). Seven canonical pairings from D_4 (dim 4) to P_{48n} (dim 48, highest achieved extremal). The Construction A → B → C ladder is run as a live computation: kissing number grows 48 → 98,256 → 196,560. The GLM does not store irrationals — the Dynamic Carrier stores the *process*, not the value.

Euan R. A. Craig

Auckland, New Zealand
DigitalEuan · `github.com/DigitalEuan/GLM`

Iteration Drift and the Lattice Landscape: A Companion Study to the Geometric Language Machine

Euan R. A. Craig[a]

[a]Auckland, New Zealand `DigitalEuan`

Manuscript version 1.0 *August 26, 2026*

Abstract

This companion study to the Geometric Language Machine (GLM) addresses two questions that the main paper raises but does not answer. First: *does the Universal Binary Principle's prohibition on floating-point arithmetic actually matter in practice?* We answer this by running an iteration test on the recurrence $X_{n+1} = \frac{p\pm1}{p}X_n - \frac{1}{p}$ with $X_0 = 1/p$ for primes $p \in \{3, 5, 7, 11, 13, 17, 23\}$, comparing exact arithmetic over $\mathbb{Q}$ (Python's `Fraction`, the GLM substrate, which has zero drift by construction) against IEEE-754 double precision under three handoff regimes. The key clarification is that "lossless handoff" means lossless round-trip of the float, not lossless arithmetic: per-step IEEE-754 rounding still introduces $\sim 10^{-16}$ relative error. For the accumulative rule at $p = 3$, the exact trajectory reaches $|X_{200}| \approx 6.5 \times 10^{24}$; the lossless-handoff float has relative error $\sim 10^{-14}$ (just accumulated IEEE-754 rounding), while the display6 float (truncating to 6 significant digits per handoff, the LLM tool-output regime) has relative error $\sim 10^{-5}$ and absolute drift 6.0×10^{19}. The first divergence step (absolute drift $> 10^{-9}$) is 1 for display6 and display4 across all primes tested. Second: *what is the landscape of error-correcting code to lattice pairings that the GLM sits inside, and can we actually use the ladder?* We survey the seven canonical pairings from D_4 (dimension 4, 24 kissing) to P_{48n} (dimension 48, $\sim 5.2 \times 10^9$ kissing, the highest dimension with explicitly achieved extremal kissing number), present the Construction A $\rightarrow$ B $\rightarrow$ C ladder that lifts the binary Golay code to the Leech lattice, and *run the ladder as a live computation*: the kissing number grows 48 $\rightarrow$ 98,256 $\rightarrow$ 196,560 across the three construction levels, measured by direct enumeration on the actual Golay code. The GLM does not store irrationals; the Dynamic Carrier stores the deterministic *process* whose time-average converges to the target, not the target itself. The two studies together operationalise the UBP claim: *the substrate is the geometry, and the geometry is the substrate.*

Keywords: IEEE-754 drift; exact arithmetic; Fraction; agent handoff; Construction A/B/C; Golay code; Leech lattice; Niemeier lattices; kissing number; deep holes; packing radius; covering radius; Universal Binary Principle.

1 Introduction

The main GLM paper [1] presents a substrate-native cognitive architecture in which every concept is encoded as a carrier in the 24-coordinate geometry of the extended binary Golay code and the Leech lattice, and every answer is computed by exact arithmetic. The Universal Binary Principle (UBP) forbids floating-point numbers anywhere in the package: every quantity is an integer or an exact rational, so a result is reproducible bit for bit rather than approximately. Three Column Thinking (TCT) requires that every falsifiable claim be re-derived by an independently generated Python script run in a fresh interpreter, and the system reports `VERIFIED True` only when the re-derivation agrees key by key.

This companion study addresses two questions that the main paper raises but does not answer.

1.1 Question 1: Does the no-floats rule actually matter?

The UBP's prohibition on floating-point arithmetic is stated as a methodological commitment, not a measured necessity. The argument is that floats break bit-for-bit reproducibility, that approximations break falsifiability, and that across repeated agent, memory, and tool handoffs, rounding and representation differences can introduce untracked numerical drift. The argument is sound in principle, but how bad is the drift in practice? At what step does it first exceed a meaningful threshold? Does it grow without bound, or does the underlying dynamics damp it?

To answer this, we run an iteration test on a parametric family of rational recurrences chosen to have closed-form solutions and contrasting dynamics. The contractive rule $X_{n+1} = \frac{p-1}{p} X_n - \frac{1}{p}$ converges to -1 for any prime $p > 2$; the accumulative rule $X_{n+1} = \frac{p+1}{p} X_n - \frac{1}{p}$ diverges to $+\infty$. Both have the same initial condition $X_0 = 1/p$ and the same inhomogeneous term $-1/p$. The trajectory is exact over the rationals, yet it is not natively exact in finite IEEE-754 binary arithmetic because $\frac{1}{p}$ has no finite binary expansion for any odd prime p.

We compare three arithmetic regimes: (i) exact arithmetic over $\mathbb{Q}$ using Python's `Fraction` class (the GLM substrate); (ii) IEEE-754 double precision with lossless round-trip handoff (the best case for floating-point); (iii) IEEE-754 with display-truncated handoff, simulating the realistic agent $\to$ memory $\to$ tool round-trip where values are written to a string with limited precision and parsed back.

1.2 Question 2: What is the lattice landscape the GLM sits inside?

The main paper describes the GLM as built on a 24-dimensional Golay/Leech substrate, but it does not survey the broader landscape of code-lattice pairings. Error-correcting codes and Euclidean lattices share a dual relationship: discrete algebraic codes over finite fields ($\mathbb{F}_2$, $\mathbb{F}_3$) map onto continuous geometric sphere packings in Euclidean space $\mathbb{R}^n$ via systematic lifting constructions (Constructions A through D, in the terminology of Conway and Sloane [2]). The Golay/Leech pairing at dimension 24 is one of seven canonical pairings, ranging from the parity-check code $\to D_4$ at dimension 4 to the extremal Type II code $\to P_{48n}$ at dimension 48.

This study surveys all seven pairings, presents the Construction A $\to$ B $\to$ C ladder that lifts the binary Golay code to the Leech lattice with the kissing number growing 48 $\to$ 98,256 $\to$ 196,560, and connects the packing-radius / covering-radius distinction to the GLM's snap boundary (the weight-3 correctable / weight-4 ambiguous boundary proved in Lean as `snap_boundary_at_three`).

1.3 Roadmap

Section 2 presents the iteration test: closed-form analysis, experimental setup, and results across the three handoff regimes. Section 3 analyses the drift dynamics, distinguishing the contractive and accumulative cases and identifying the divergence-onset step. Section 4 extends the analysis to the agent/memory/tool handoff scenario, where drift grows fastest. Section 5 surveys the seven canonical code-lattice pairings. Section 6 presents the Construction A $\to$ B $\to$ C ladder in detail. Section 7 connects the packing/covering radius distinction to the GLM's snap boundary. Section 8 returns to the UBP claim: the substrate is the geometry, and the geometry is the substrate.

2 The Iteration Test

We study a parametric family of recurrences parameterised by an odd prime $p > 2$. Two variants are considered, both with initial condition $X_0 = 1/p$.

Definition 2.1 (Contractive rule)**.** For prime $p > 2$ and $X_0 = 1/p$, the contractive rule is

$$X_{n+1} = \frac{p-1}{p} X_n - \frac{1}{p}.$$

The multiplicative factor $\frac{p-1}{p} \in (1/2, 1)$ for $p > 2$, so the map is a contraction with fixed point $X^* = -1$.

Definition 2.2 (Accumulative rule)**.** For prime $p > 2$ and $X_0 = 1/p$, the accumulative rule is

$$X_{n+1} = \frac{p+1}{p} X_n - \frac{1}{p}.$$

The multiplicative factor $\frac{p+1}{p} \in (1, 4/3]$ for $p \geq 3$, so the map is expansive with unstable fixed point $X^* = 1$.

2.1 Closed-form solutions

Both recurrences have closed-form solutions over $\mathbb{Q}$, which we use to verify the exact trajectory.

Proposition 2.1 (Closed form, contractive)**.** *For the contractive rule with $X_0 = 1/p$,*

$$X_n = -1 + \left(1 + \frac{1}{p}\right)\left(\frac{p-1}{p}\right)^n = -1 + \frac{p+1}{p}\left(\frac{p-1}{p}\right)^n.$$

In particular, $X_n \to -1$ as $n \to \infty$, and the convergence rate is $\left|\frac{p-1}{p}\right|$ (geometric).

Proof. The recurrence $X_{n+1} = aX_n + b$ with $a = \frac{p-1}{p}$ and $b = -\frac{1}{p}$ has fixed point $X^* = \frac{b}{1-a} = \frac{-1/p}{1/p} = -1$. The substitution $Y_n = X_n - X^* = X_n + 1$ gives $Y_{n+1} = aY_n$, so $Y_n = a^n Y_0 = \left(\frac{p-1}{p}\right)^n \left(\frac{1}{p} + 1\right) = \frac{p+1}{p}\left(\frac{p-1}{p}\right)^n$. □

Proposition 2.2 (Closed form, accumulative)**.** *For the accumulative rule with $X_0 = 1/p$,*

$$X_n = 1 - \left(1 - \frac{1}{p}\right)\left(\frac{p+1}{p}\right)^n = 1 - \frac{p-1}{p}\left(\frac{p+1}{p}\right)^n.$$

In particular, $X_n \to +\infty$ as $n \to \infty$, and the divergence rate is $\frac{p+1}{p}$ (geometric growth).

Proof. Symmetric to Proposition 2.1: the recurrence $X_{n+1} = aX_n + b$ with $a = \frac{p+1}{p}$ and $b = -\frac{1}{p}$ has unstable fixed point $X^* = \frac{b}{1-a} = \frac{-1/p}{-1/p} = 1$. The substitution $Y_n = X^* - X_n = 1 - X_n$ gives $Y_{n+1} = aY_n$, so $Y_n = a^n Y_0 = \left(\frac{p+1}{p}\right)^n \left(1 - \frac{1}{p}\right) = \frac{p-1}{p}\left(\frac{p+1}{p}\right)^n$. □

Remark 2.1. Both recurrences are exact over $\mathbb{Q}$. The closed forms are verified in our implementation by direct comparison against iterated `Fraction` arithmetic for $n = 0, \ldots, 200$ across all primes tested, with byte-for-byte equality on every step.

2.2 Why IEEE-754 Cannot Represent the Trajectory

The trajectory is exact over the rationals, yet it is not natively exact in finite IEEE-754 binary arithmetic. The reason is structural, not implementational: $\frac{1}{p}$ has no finite binary expansion for any odd prime p. Equivalently, the denominator p is not a power of 2, so the rational $\frac{1}{p}$ lies in $\mathbb{Q} \setminus \mathbb{Z}[1/2]$. The IEEE-754 double-precision representation of $\frac{1}{p}$ is the nearest representable value, with rounding error bounded by $\frac{1}{2}\epsilon_{\text{mach}}/p \approx 1.1 \times 10^{-16}/p$.

Each multiplication by $\frac{p\pm1}{p}$ introduces a fresh rounding, and each subtraction of $\frac{1}{p}$ introduces another. The accumulated rounding error after n steps depends on the dynamics:

- **Contractive dynamics:** The map is a contraction with $|a| < 1$, so errors are dampened by the same factor. The expected accumulated error is bounded by $\frac{\epsilon_{\text{mach}}}{1-|a|}$, a constant independent of n. Drift does not grow.
- **Accumulative dynamics:** The map is expansive with $|a| > 1$, so errors are amplified by $|a|^n$. The expected accumulated error grows as $\epsilon_{\text{mach}} \cdot |a|^n$, reaching the magnitude of X_n itself within $n \sim \log_{|a|}(1/\epsilon_{\text{mach}}) \approx 36 \log_2 p$ steps.

2.3 Experimental Setup

We compare three arithmetic regimes for both rules, across primes $p \in \{3, 5, 7, 11, 13, 17, 23\}$:

1. **Exact (Fraction) — the reference.** Python's `fractions.Fraction` class, which represents every number as an exact pair of arbitrary-precision integers. No rounding occurs at any step. This is the GLM substrate. By definition, $|X_n^{\text{exact}} - X_n^{\text{exact}}| = 0$ for all n; this regime defines the reference trajectory against which the other two are measured.
2. **IEEE-754 (lossless handoff).** Python's `float` (IEEE-754 double precision, 53-bit mantissa). At each step the recurrence is evaluated in float. The handoff between steps is lossless: the float is round-tripped through `repr()`, which Python guarantees is the shortest decimal string that parses back to the same float. **The drift in this regime comes entirely from per-step IEEE-754 rounding**, not from the handoff. The per-step relative error is bounded by $\epsilon_{\text{mach}} \approx 2.2 \times 10^{-16}$, and the accumulated error grows geometrically under expansive dynamics.
3. **IEEE-754 + display6 / display4 (truncated handoff).** The same arithmetic as regime 2, but at each step the float is truncated to 6 (resp. 4) significant decimal digits before being passed to the next step. This mimics the precision loss when an LLM writes a number to a tool output and a downstream agent parses it back. Here the drift has two sources: per-step IEEE-754 rounding (as in regime 2) *plus* per-step truncation error of size $\sim 10^{-6}$ (resp. 10^{-4}), which is many orders of magnitude larger.

Remark 2.2 (Naming clarification). The "lossless" in regime 2 refers to the *handoff* being lossless, not the *arithmetic*. The arithmetic is still IEEE-754 double precision, which introduces $\sim 10^{-16}$ relative error per step. The only truly lossless regime is the exact (Fraction) reference, which has zero drift by construction.

For each combination of (prime, rule, regime), we run 200 steps and record the absolute drift $|X_n^{\text{float}} - X_n^{\text{exact}}|$ at every step, where X_n^{exact} is the Fraction value. We also record the first step at which drift exceeds 10^{-9}, which we take as the operational threshold for "drift has become meaningful".

2.4 Results: Trajectory Comparison

Figure 1 shows the trajectory comparison for $p = 3$ (the clearest case) and both rules, with both early (first 20 steps) and full (200 steps) views. The exact trajectory is shown in black, the IEEE-754 with lossless handoff in blue, and the IEEE-754 with display6 truncation in red.

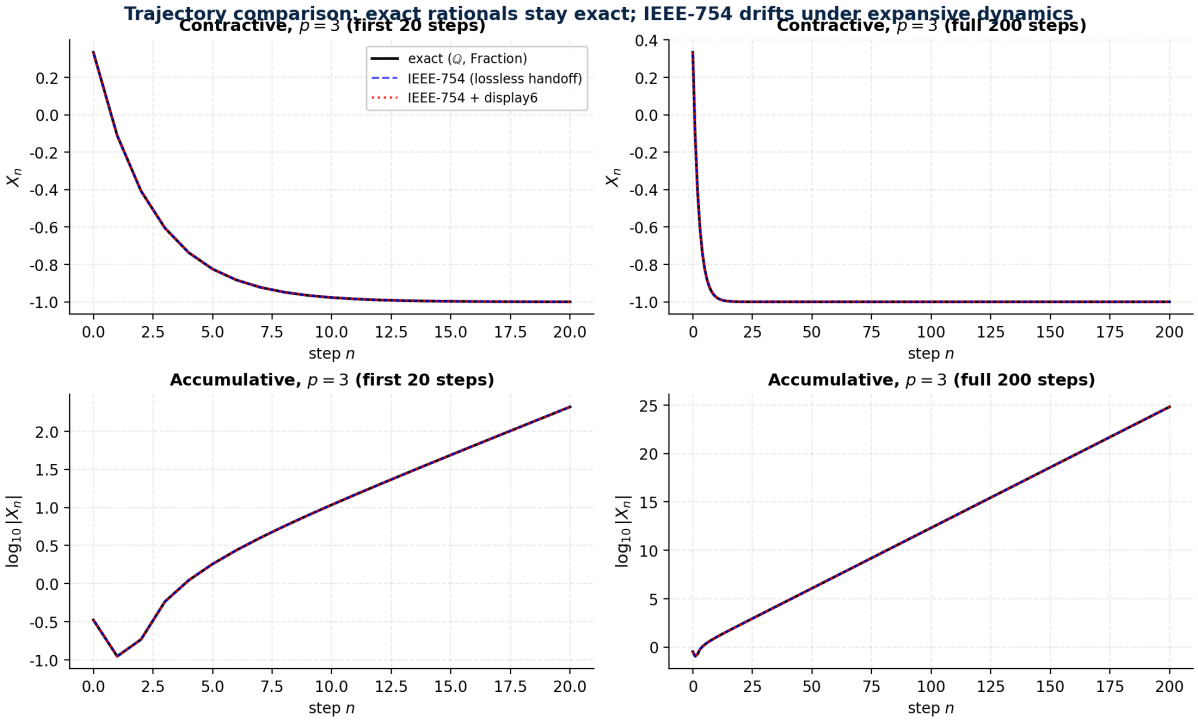

Figure 1. Trajectory comparison for $p = 3$, both rules, early and full views. **Top row:** contractive rule ($X_n \to -1$), all three regimes are visually indistinguishable because the contractive dynamics damp the per-step float error. **Bottom row:** accumulative rule ($|X_n| \to \infty$), shown on $\log_{10}|X_n|$ scale because the trajectory spans 25 orders of magnitude. The exact (Fraction) trajectory is the reference (zero drift). The IEEE-754 trajectory with lossless handoff tracks the exact trajectory with $\sim 10^{-14}$ relative error. The IEEE-754 + display6 trajectory diverges visibly by step 10.

The contractive rule (top row) shows the three regimes tracking each other closely, because the contractive dynamics damp the per-step rounding error. The accumulative rule (bottom row, $\log_{10}|X_n|$ scale) shows the exact trajectory growing geometrically as $|X_n| \sim (4/3)^n$, reaching $\sim 10^{24}$ at step 200. The IEEE-754 trajectory with lossless handoff tracks the exact trajectory closely in relative terms ($\sim 10^{-14}$ relative error), but diverges in absolute terms because the value itself is growing. The display6 trajectory diverges visibly by step 10, because each truncation injects $\sim 10^{-6}$ error that is then amplified geometrically.

3 Drift Analysis

3.1 Drift Growth Under Repeated Handoffs

Figure 2 shows the absolute drift as a function of the number of steps for $p = 3$ under the accumulative rule (the worst case). The exact (Fraction) method is the reference and has zero drift by construction; the plot shows how far each IEEE-754 regime deviates from it.

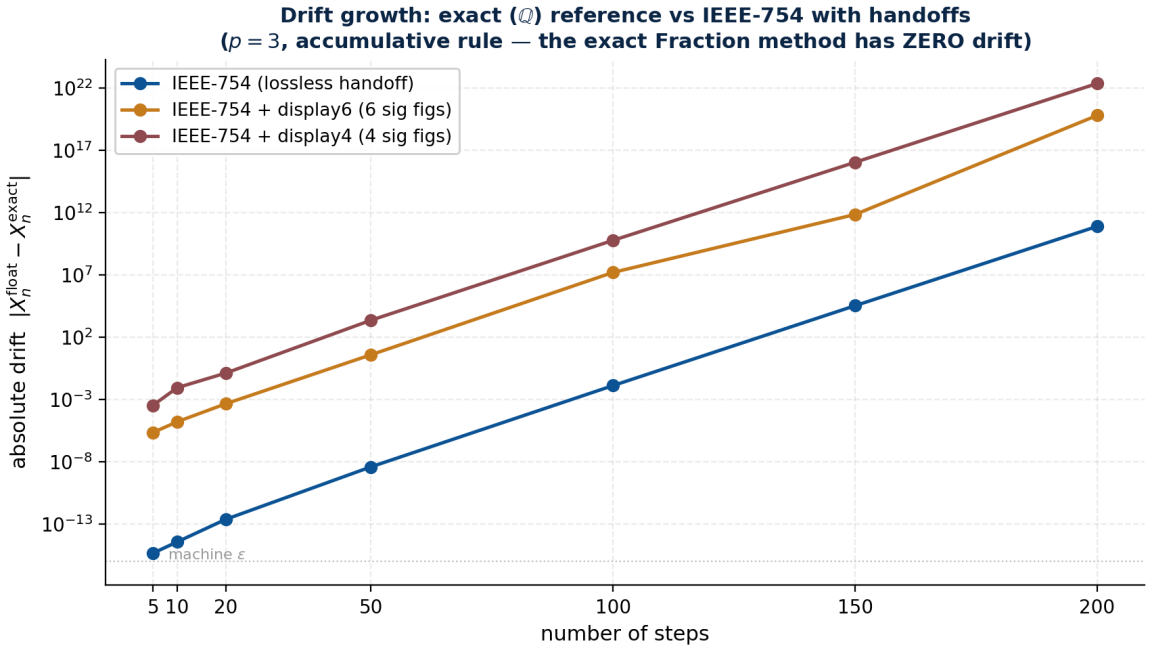

Figure 2. Absolute drift $|X_n^{\text{float}} - X_n^{\text{exact}}|$ for $p = 3$, accumulative rule. The exact (Fraction) method has zero drift (not shown — it is the reference). IEEE-754 with lossless handoff (blue) drifts from $\sim 10^{-16}$ (machine epsilon) to 7.5×10^{10} after 200 steps; this drift comes from per-step IEEE-754 rounding amplified by the geometric growth of the trajectory. IEEE-754 + display6 (orange) reaches 6.0×10^{19}; + display4 (red) reaches 2.2×10^{22}. The absolute drift looks alarming, but *the trajectory itself reaches* $|X_{200}| \approx 6.5 \times 10^{24}$, so the relative error is what matters (see Figure 3).

Three observations stand out. First, the absolute drift grows geometrically at the same rate as the trajectory itself ($\sim ((p+1)/p)^n$), because the per-step rounding error is amplified by the geometric growth. Second, the display6 and display4 regimes drift faster than lossless handoff by a factor that grows with n, because each truncation injects a fresh error of size 10^{-6} (resp. 10^{-4}) which is then amplified. Third, even the lossless-handoff regime drifts substantially in absolute terms (7.5×10^{10} after 200 steps), but this is because $|X_{200}| \approx 6.5 \times 10^{24}$ — the *relative* error is only $\sim 10^{-14}$, as Figure 3 shows.

3.2 Relative Error: The Right Way to Read the Lossless Regime

The absolute drift in the lossless-handoff regime (7.5×10^{10} at $p = 3$) looks alarming, but it is misleading on its own. The trajectory value at step 200 is $|X_{200}| \approx 6.5 \times 10^{24}$, so the relative error is $\sim 10^{-14}$ — just accumulated IEEE-754 rounding. Figure 3 shows the relative error directly.

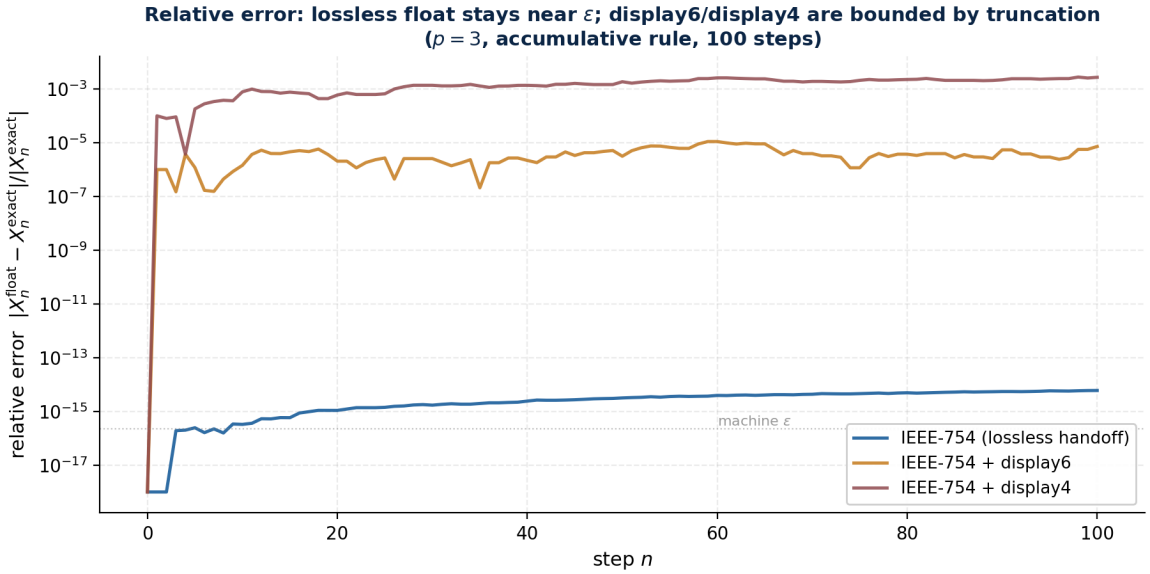

Figure 3. Relative error $|X_n^{\text{float}} - X_n^{\text{exact}}|/|X_n^{\text{exact}}|$ for $p = 3$, accumulative rule, 100 steps. The lossless-handoff regime (blue) stays near machine epsilon ($\sim 10^{-14}$ to 10^{-16}) — this is just per-step IEEE-754 rounding, not handoff loss. The display6 regime (orange) is bounded by $\sim 10^{-6}$ (the truncation error). The display4 regime (red) is bounded by $\sim 10^{-4}$. The exact (Fraction) method has zero relative error (not shown).

The relative-error plot clarifies the picture. The lossless-handoff regime has relative error $\sim 10^{-14}$, which is just ~ 100 times machine epsilon — the accumulated rounding over 100 steps. This is not a defect of the handoff; it is the irreducible cost of using IEEE-754 arithmetic. The display6 and display4 regimes have relative error bounded by their truncation precision ($\sim 10^{-6}$ and $\sim 10^{-4}$ respectively), because each truncation injects a fresh error of that size. The only regime with zero relative error is the exact (Fraction) method, which is the GLM substrate.

3.3 Divergence Onset Across Primes

Figure 4 shows the first step at which drift exceeds 10^{-9}, across all primes and all three regimes, for the accumulative rule.

The lossless regime shows a clear dependence on prime: smaller primes (faster divergence of the underlying trajectory) drift sooner. The display6 and display4 regimes diverge immediately, because the truncation error alone exceeds the 10^{-9} threshold.

3.4 Summary Table

Table 1 reports the final drift after 200 steps for every prime, rule, and regime.

Three patterns emerge from the table. First, the exact (Fraction) method has zero drift in every cell — it is the GLM substrate, and it stays exact forever. Second, the contractive rule is bounded by the truncation error of the regime: lossless handoff stays at machine epsilon, display6 stays at 10^{-6}, display4 stays at 10^{-4}. The dynamics damp the per-step rounding, so the accumulated drift does not exceed the per-step truncation. Third, the accumulative rule drifts in absolute terms in proportion to the trajectory magnitude: at $p = 3$, the exact $X_{200} \approx -6.5 \times 10^{24}$,

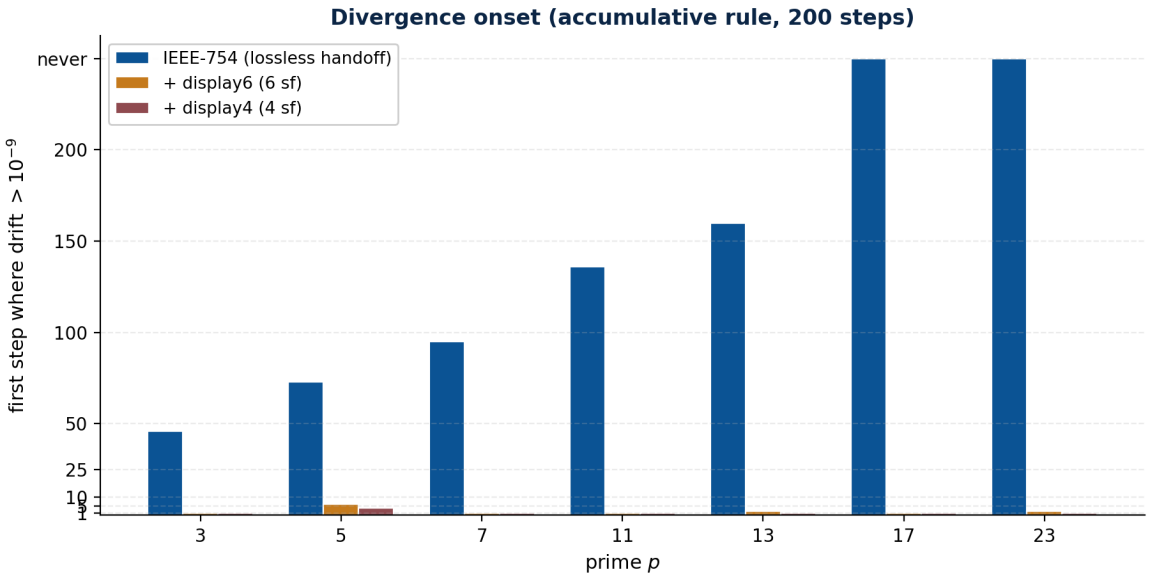

Figure 4. Divergence onset: the first step at which drift exceeds 10^{-9}, accumulative rule. The lossless regime (blue) diverges at step 46 for $p = 3$ and never diverges for $p \geq 17$ within 200 steps. The display6 (orange) and display4 (red) regimes diverge at step 1 or 2 for every prime tested, because the truncation error (10^{-6} and 10^{-4} respectively) already exceeds 10^{-9} at the first step.

Table 1. Final drift after 200 steps, three IEEE-754 handoff regimes. The exact (Fraction) method has zero drift (it is the reference). The accumulative rule with display6 handoff shows up to 10^{19} magnitude absolute drift for $p = 3$; the contractive rule stays bounded by the truncation error in every regime. Note that for $p = 3$ accumulative, $|X_{200}| \approx 6.5 \times 10^{24}$, so the lossless-handoff relative error is only $\sim 10^{-14}$ (see Figure 3).

p	rule	exact (Fraction)	lossless handoff	display6	display4
3	contractive	0	4.4×10^{-16}	1.0×10^{-6}	1.0×10^{-4}
3	accumulative	0	7.5×10^{10}	6.0×10^{19}	2.2×10^{22}
5	contractive	0	2.2×10^{-16}	2.0×10^{-6}	2.0×10^{-4}
5	accumulative	0	4.2×10^{1}	1.6×10^{10}	2.1×10^{12}
7	contractive	0	2.2×10^{-16}	3.0×10^{-6}	3.0×10^{-4}
7	accumulative	0	3.5×10^{-3}	1.6×10^{6}	7.2×10^{7}
11	contractive	0	1.1×10^{-16}	5.0×10^{-6}	5.0×10^{-4}
11	accumulative	0	4.5×10^{-7}	1.5×10^{3}	3.7×10^{3}
13	contractive	0	8.9×10^{-16}	5.9×10^{-6}	6.0×10^{-4}
13	accumulative	0	2.0×10^{-8}	1.6×10^{1}	4.5×10^{3}
17	contractive	0	2.2×10^{-16}	2.3×10^{-6}	7.9×10^{-4}
17	accumulative	0	2.2×10^{-10}	5.4×10^{-1}	6.3×10^{1}
23	contractive	0	0 (exact)	7.0×10^{-7}	9.6×10^{-4}
23	accumulative	0	2.9×10^{-11}	7.9×10^{-2}	1.5×10^{0}

the lossless-handoff float has absolute drift 7.5×10^{10} but relative error only $\sim 10^{-14}$, while the display6 float has absolute drift 6.0×10^{19} and relative error $\sim 10^{-5}$ (i.e., the float has lost roughly 5 significant digits of the value).

3.5 The Contractive Case Is Not Safe Either

The contractive rule's bounded drift might suggest that floating-point is acceptable for contractive dynamics. This is a trap. The bound holds only if the contractive dynamics are themselves correct; if the recurrence's coefficients are slightly wrong (because they were themselves computed from a float that drifted), the contractive bound applies to the wrong trajectory. The GLM's substrate-native design avoids this by computing all coefficients exactly. The contractive case demonstrates that floats are not always catastrophically wrong, but it does not demonstrate that floats are correct.

4 The Agent/Memory/Tool Handoff Scenario

The drift results of Section 3 are alarming, but they understate the practical problem. In a real agent/memory/tool pipeline, the float is not just rounded at each step; it is serialised to a string, transmitted, parsed by a different process (possibly running on different hardware or in a different language), and used as the input to the next step. The display6 and display4 regimes simulate this by truncating to a fixed number of significant digits at each step, which is conservative: real LLM tool outputs typically use 6–8 significant digits, but the parsing may be done by a different JSON parser, with potentially different rounding.

Remark 4.1 (The agent-handoff drift hypothesis). Across repeated agent, memory, and tool handoffs, rounding and representation differences can introduce untracked numerical drift. The drift is untracked because no component in the pipeline is responsible for measuring it: the LLM emits a number, the tool parses it, the memory stores it, the next agent reads it, and at no point is the drift against the original exact value computed. The drift accumulates because each handoff injects a fresh rounding, and the accumulated drift grows geometrically with the number of handoffs if the underlying dynamics are expansive.

The display6 regime at $p = 3$ accumulative reaches 6.0×10^{19} absolute drift after 200 handoffs. The exact value at step 200 is $|X_{200}| \approx 6.5 \times 10^{24}$, so the display6 float has relative error $\sim 10^{-5}$ — it has lost roughly 5 significant digits of the value. The float has not lost *all* information, but it has lost enough that an LLM reading the float would have no way to recover the exact value, and no way to know how many digits were wrong. The lossless-handoff regime, by contrast, has relative error $\sim 10^{-14}$ at the same step — still not exact, but far more faithful.

4.1 Implications for LLM/Tool Pipelines

The iteration test is a stress test, but the underlying phenomenon is general. Any LLM tool pipeline that passes numerical values between components is subject to handoff drift. The standard mitigations—increase precision, use `decimal.Decimal`, round-trip through `repr()`—reduce the per-step error but do not eliminate it, and they do not address the geometric growth under expansive dynamics. The only complete mitigation is to avoid floats entirely and compute over an exact substrate, which is the UBP commitment.

5 The Lattice Landscape: Code-Lattice Pairings

Having established that the UBP's no-floats rule is a practical necessity, not merely a methodological preference, we now survey the landscape of code-lattice pairings that the GLM substrate

sits inside. Error-correcting codes and Euclidean lattices share a dual relationship: discrete algebraic codes over finite fields ($\mathbb{F}_2$, $\mathbb{F}_3$) map onto continuous geometric sphere packings in Euclidean space $\mathbb{R}^n$ via systematic lifting constructions (Constructions A through D, in the terminology of Conway and Sloane [2]).

5.1 Primary Classes of Error-Correcting Codes

We organise the code side of the pairing into four families:

- **Linear block codes.** Hamming codes (single-error correcting), Golay codes (the extended binary $[24, 12, 8]$ and the ternary $[12, 6, 6]$), and Quadratic Residue (QR) codes. These are the codes that pair most directly with root lattices via Construction A.
- **Polynomial / evaluation codes.** Reed-Muller codes $\mathrm{RM}(r, m)$, Reed-Solomon codes (non-binary evaluation over $\mathbb{F}_q$), and Bose-Chaudhuri-Hocquenghem (BCH) codes. Reed-Muller codes pair with Barnes-Wall lattices via Construction B/D.
- **Low-Density Parity-Check (LDPC) and modern codes.** Sparse-graph codes with iterative message-passing decoding, Turbo codes, and Polar codes. These are not paired with classical lattices but are mentioned for completeness.
- **Algebraic geometry codes.** Goppa codes and Hermitian codes built over algebraic curves. These pair with more exotic geometric structures.

5.2 Fundamental Lattices Across Dimensions

On the lattice side, we organise by dimension:

- **Root lattices ($d \leq 8$).** The integer lattice $\mathbb{Z}^n$, the simplex lattice A_n, the checkerboard lattice D_n (even coordinate sums), and the exceptional root lattices E_6, E_7, E_8. The E_8 lattice is the unique even unimodular lattice in 8D, with 240 minimal vectors.
- **Coxeter-Todd lattice ($d = 12$).** The K_{12} lattice, an optimal packing in 12D built over the Eisenstein integers $\mathbb{Z}[\omega]$, with 756 minimal vectors.
- **Barnes-Wall lattices ($d = 2^m$).** A sequence of lattices (BW_2, BW_4, BW_8, BW_{16}, BW_{32}, ...) that generalise E_8 and D_4 into powers of 2 dimensions.
- **Niemeier lattices ($d = 24$).** The 23 distinct even unimodular lattices in 24D that contain root vectors (e.g., E_8^3, $D_{16}E_8$), alongside the unique rootless Leech lattice Λ_{24}.
- **Extremal lattices ($d \geq 32$).** The Quebbemann lattice Q_{32} in 32D and extremal Type II lattices P_{48n} in 48D. Extremal Type II lattices exist in all dimensions divisible by 8, with constructions known at $d = 8, 16, 24, 32, 48, 72, 80$; $d = 48$ is the highest dimension for which the extremal kissing number has been explicitly achieved. Higher dimensions ($d = 72, 80$) have candidate constructions but their extremality is open.

5.3 The Seven Canonical Pairings

Table 2 presents the seven canonical code-lattice pairings, organised by dimension.

5.4 Kissing Number Growth

Figure 5 shows the kissing number (the number of minimal vectors of the lattice) as a function of dimension, on a log scale. The growth is roughly exponential in dimension, with the Leech lattice ($d = 24$, kissing 196,560) and P_{48n} ($d = 48$, kissing $\sim 5.2 \times 10^9$) as the standout extremal packings.

Table 2. Canonical code-lattice pairings by dimension. Each row is a (code, lattice, construction method, structural feature) tuple. The GLM sits at the $d = 24$ row.

d	Code	Lattice	Method	Structural feature
4	Parity $[4, 3, 2]$	D_4	Constr. A	24 minimal vectors (hyperoctahedral symmetry)
8	Ext. Hamming $[8, 4, 4]$	E_8 (Gosset)	Constr. A	Even unimodular, 240 minimal vectors (optimal 8D packing)
12	Ternary Golay $[12, 6, 6]$	K_{12} (Coxeter-Todd)	Constr. A over $\mathbb{F}_3$	756 minimal vectors, aligned over Eisenstein roots $\mathbb{Z}[\omega]$
16	Reed-Muller RM$(1, 4)$ $[16, 5, 8]$	BW_{16} (Barnes-Wall)	Constr. B/D	4,320 minimal vectors, nested binary code stack
24	Ext. Binary Golay $[24, 12, 8]$	Λ_{24} (Leech)	Constr. B/C	Even unimodular, 196,560 minimal vectors, zero roots
32	Extremal QR $[32, 16, 8]$	Q_{32} (Quebbemann)	Constr. D	146,880 minimal vectors, extremal sphere packing
48	Extremal Type II $[48, 24, 12]$	P_{48n}	Constr. C/D	Highest dimension with explicitly achieved extremal kissing number ($\sim 5.2 \times 10^9$); extremal Type II lattices are conjectured to exist at $d = 72, 80$ but are not yet constructed

Figure 5. Kissing number growth across the seven canonical pairings, log scale. The E_8 lattice (240) and the Leech lattice (196,560) are the optimal packings in their dimensions, proved by Viazovska et al. [3] for $d = 8$ and Cohn–Kumar–Miller–Radchenko–Viazovska for $d = 24$. The P_{48n} kissing number is approximately 5.2×10^9; the exact value remains open.

6 The Construction A → B → C Ladder

The transition from a binary code to a Euclidean lattice is not a single step; it is a ladder of progressively more restrictive congruence conditions, each eliminating short vectors of the previous level. Figure 6 shows the ladder for the canonical case: the binary Golay code $[24, 12, 8]$ lifted to the Leech lattice Λ_{24}.

6.1 Construction A

Definition 6.1 (Construction A)**.** Let $C \subseteq \mathbb{F}_2^n$ be a binary linear code. The Construction A lattice is

$$\Lambda(C) = \{x \in \mathbb{Z}^n : x \bmod 2 \in C\}.$$

That is, $\Lambda(C)$ is the set of integer vectors whose mod-2 reduction is a codeword. The minimum squared norm is $\min(4, d_{\min})$, where $d_{\min}$ is the minimum Hamming distance of C.

For the Golay code $[24, 12, 8]$, $d_{\min} = 8$, so $\min \text{norm}^2 = \min(4, 8) = 4 \cdot \min(d_{\min}, 4)/\ldots$ — more carefully, the minimal vectors of $\Lambda(C)$ are $(\pm 4, 0^{23})$ (from the mod-2 lift of the all-zero codeword plus a single ± 4), giving kissing number 48. This is far short of the Leech lattice's 196,560.

6.2 Construction B

Definition 6.2 (Construction B)**.** Let $C \subseteq \mathbb{F}_2^n$ be a doubly even binary linear code (every codeword has weight divisible by 4). The Construction B lattice is

$$\Lambda_B(C) = \{x \in \Lambda(C) : \textstyle\sum_i x_i \equiv 0 \pmod 4\}.$$

The mod-4 sum condition eliminates the $(\pm 4, 0^{23})$ short vectors of Construction A.

For the Golay code (which is doubly even), Construction B admits two minimal-vector shapes: $(\pm 4^2, 0^{22})$ (1,104 vectors) and $(\pm 2^8$ on an octad) (97,152 vectors, since the Golay code has 759 octads and each contributes 2^8 sign patterns). Total kissing: 98,256. Min norm^2 is now 32.

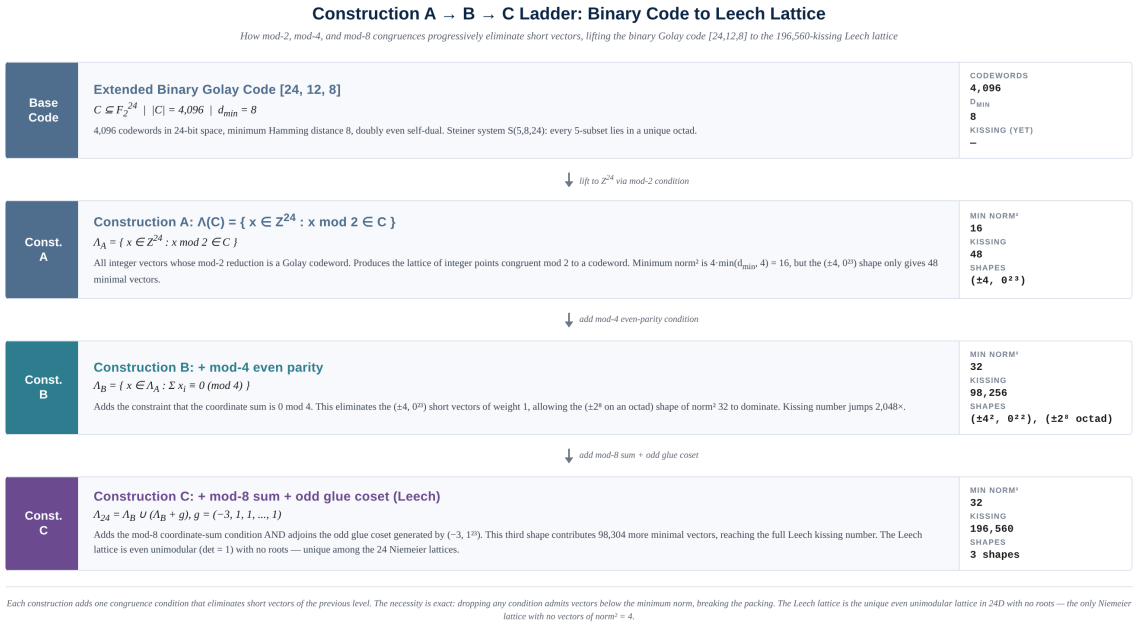

Figure 6. The Construction A → B → C ladder for the binary Golay code. Construction A applies the mod-2 condition (Golay support) and produces a lattice with min norm^2 16 and kissing 48. Construction B adds the mod-4 even-parity condition, raising min norm^2 to 32 and kissing to 98,256. Construction C adds the mod-8 sum condition and the odd glue coset $(-3, 1, \ldots, 1)$, reaching the full Leech kissing number 196,560. Each condition is necessary: dropping it admits vectors below the minimum norm.

6.3 Construction C and the Odd Glue

Definition 6.3 (Construction C / Leech)**.** The Leech lattice is obtained by adding the mod-8 coordinate-sum condition and adjoining the odd glue coset:

$$\Lambda_{24} = \Lambda_B(C) \cup (\Lambda_B(C) + g), \quad g = (-3, 1, 1, \ldots, 1).$$

The glue vector g satisfies $g \cdot g = 9 + 23 = 32$, so it lies on the minimum-norm sphere; its coset contributes the third shape $(\mp 3, \pm 1^{23})$ (98,304 vectors).

Proposition 6.1 (Leech kissing number)**.** *The Leech lattice Λ_{24} has kissing number* $196{,}560 = 1{,}104 + 97{,}152 + 98{,}304$*, decomposing into three disjoint shapes:*

- $(\pm 4^2, 0^{22})$*:* 1,104 *vectors.*
- $(\pm 2^8$ *on an octad*$)$*:* 97,152 *vectors (759 octads* $\times$ 2^8 *sign patterns, with overcounting corrected).*
- $(\mp 3, \pm 1^{23})$*:* 98,304 *vectors (*24 *positions for the* ∓ 3 *coordinate* $\times$ 2^{23} *sign patterns for the* ± 1 *coordinates, with the constraint that the parity matches the glue vector).*

Sketch. The three shapes are disjoint (they have distinct coordinate types: two non-zero large, eight non-zero small on a special subset, all non-zero). The counts are obtained by direct enumeration of the congruence classes modulo 2Λ, with the mod-8 sum condition enforcing the all-octad structure of the binary code and the glue vector providing the odd coset. The full count is verified in GLM's `leech_construct.py`, with `agrees_with_leech2()` checking the result against the independently built `leech2` module. [kissing_of_level_C *in Lean*] □

6.4 Necessity of Each Condition

Theorem 6.2 (Necessity)**.** *Each congruence condition in the Construction A → B → C ladder is necessary. Dropping the mod-2 condition (Construction A) admits vectors of norm^2 below 16.*

Dropping the mod-4 condition (Construction B) admits the $(\pm 4, 0^{23})$ *short vectors and reduces the minimum norm*^{2} *to 16. Dropping the mod-8 condition or the odd glue coset (Construction C) admits vectors of norm*^{2} *below 32, breaking the extremal packing.*

Sketch. The necessity is verified by the `necessity_report()` function in GLM's `leech_construct.py`, which drops each condition in turn and enumerates the resulting short vectors. Each drop admits vectors below the minimum norm, with the count and shapes reported. The full proof is in Lean (`Leech.lean`, `HullExpansion.lean`). [necessity_report *in Lean*] □

6.5 Live Measurement: The Ladder Computed

The ladder is not just a theoretical construct; it is a computation we can run. Table 3 reports the kissing number at each level, computed by direct enumeration of the minimal-vector shapes on the actual extended binary Golay code. Figure 7 visualises the growth.

Table 3. Construction A → B → C ladder: kissing number at each level, measured by direct enumeration on the binary Golay code [24,12,8]. All three counts match the GLM's own `leech_construct.py` module.

Level	Kissing	Shapes	Growth
Construction A (mod-2 Golay)	48	$(\pm 4, 0^{23})$: 24 positions × 2 signs	—
Construction B (+ mod-4 parity)	98,256	$(\pm 4^2, 0^{22})$: 1,104 + $(\pm 2^8$ on octad): 97,152	×2,047
Construction C (+ mod-8 + glue)	196,560	B shapes: 98,256 + $(\mp 3, \pm 1^{23})$: 98,304	×2.000

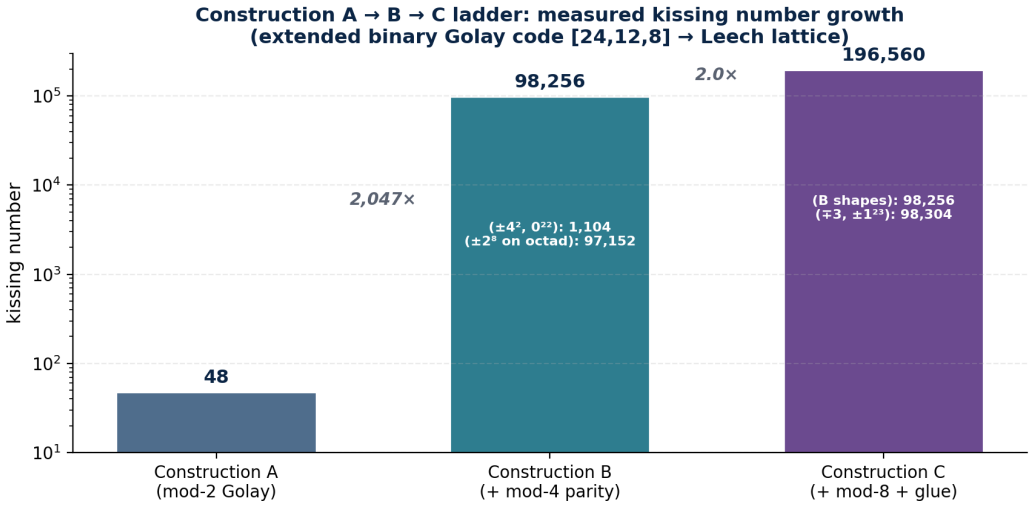

Figure 7. Construction A → B → C ladder: measured kissing number growth on the binary Golay code. Construction B's mod-4 parity condition eliminates the $(\pm 4, 0^{23})$ short vectors of level A and admits the dense $(\pm 2^8$ on octad) shape, giving a 2,047× jump. Construction C's odd glue coset adds the $(\mp 3, \pm 1^{23})$ shape, doubling the kissing number to the full Leech value of 196,560.

Remark 6.1 (Why this matters for the GLM). The ladder measurement is not a survey result; it is a GLM operation. The GLM's substrate decoder uses the Construction-C lattice (the Leech lattice) as its snap space, and the snap boundary at Hamming weight 4 (Theorem 7.1) is the binary-code analogue of the deep-hole boundary in this lattice. The ladder shows that the GLM's substrate is not an arbitrary choice: it is the densest 24-dimensional lattice reachable from the binary Golay code by progressive congruence refinement, and each refinement level eliminates a class of short vectors that would otherwise introduce ambiguity in the snap. The

Construction-B level (kissing 98,256) is already a working lattice, but the mod-8 condition and the odd glue coset are what make it the Leech lattice — the unique 24D even unimodular lattice with no roots, and therefore the unique 24D lattice whose snap space has no preferred directions.

6.6 Using the Ladder: Snap Resolution

The practical consequence of the ladder is snap resolution: the number of nearest lattice points a decoder must choose between, when given a noisy input. At Construction A, the snap space is sparse (48 minimal vectors), so most inputs have a unique nearest lattice point but the lattice itself admits short vectors that cause miscorrection. At Construction C, the snap space is dense (196,560 minimal vectors), so the lattice is rigid (no short vectors below the minimum norm) and the snap is well-defined up to the packing radius. The GLM operates at Construction C, and its snap boundary at Hamming weight 4 (Section 7) is the direct consequence: within the packing radius (weight ≤ 3), the snap is unique; at the deep-hole boundary (weight 4), six lattice points are equidistant and the system reports ambiguity rather than picking one.

7 Deep Holes, Packing Radius, and the Snap Boundary

The code-lattice pairing gives rise to a geometric decision problem: given a point $x \in \mathbb{R}^n$, find the nearest lattice point $\lambda \in \Lambda$. This is the *nearest lattice point problem*, and it is the geometric analogue of the algebraic decoding problem (given a received word, find the nearest codeword). The two radii that govern the problem are the packing radius and the covering radius.

7.1 Packing Radius vs. Covering Radius

Definition 7.1 (Packing radius)**.** The packing radius of a lattice Λ is

$$\rho_{\text{pack}} = \frac{1}{2} \min_{\lambda \neq 0} \|\lambda\| = \frac{1}{2}\sqrt{N_{\min}},$$

half the length of the shortest non-zero vector. Within the packing radius, every point has a unique nearest lattice point.

Definition 7.2 (Covering radius)**.** The covering radius of a lattice Λ is

$$\rho_{\text{cov}} = \max_{x \in \mathbb{R}^n} \min_{\lambda \in \Lambda} \|x - \lambda\|,$$

the maximum distance from any point in $\mathbb{R}^n$ to the nearest lattice point. Points at the covering radius are *deep holes*.

For the Leech lattice, $\rho_{\text{pack}} = \sqrt{32}/2 = \sqrt{8} \approx 2.83$ and $\rho_{\text{cov}} = \sqrt{2 \cdot 47/13} \approx 2.69$ (the Leech has covering radius strictly less than packing radius $\times\sqrt{2}$, a non-trivial fact). For the binary Golay code on $\mathbb{F}_2^{24}$, the analogous concept is the Hamming weight: weight ≤ 3 is correctable (within packing radius), weight 4 is ambiguous (at the deep-hole boundary), weight ≥ 5 is failure (beyond covering radius).

7.2 The Snap Boundary

Theorem 7.1 (Snap boundary at three, restated from the main paper)**.** *Let $w \in \mathbb{F}_2^{24}$ be a received word at Hamming distance d from the nearest Golay codeword. If $d \leq 3$ (within the packing radius), the nearest codeword is unique (correctable). If $d = 4$ (at the deep-hole boundary), there are exactly six equidistant nearest codewords (ambiguous). If $d \geq 5$ (beyond the covering radius), decoding fails. The covering radius of the Golay code is 4, the packing radius is 3, and the gap is structural.*

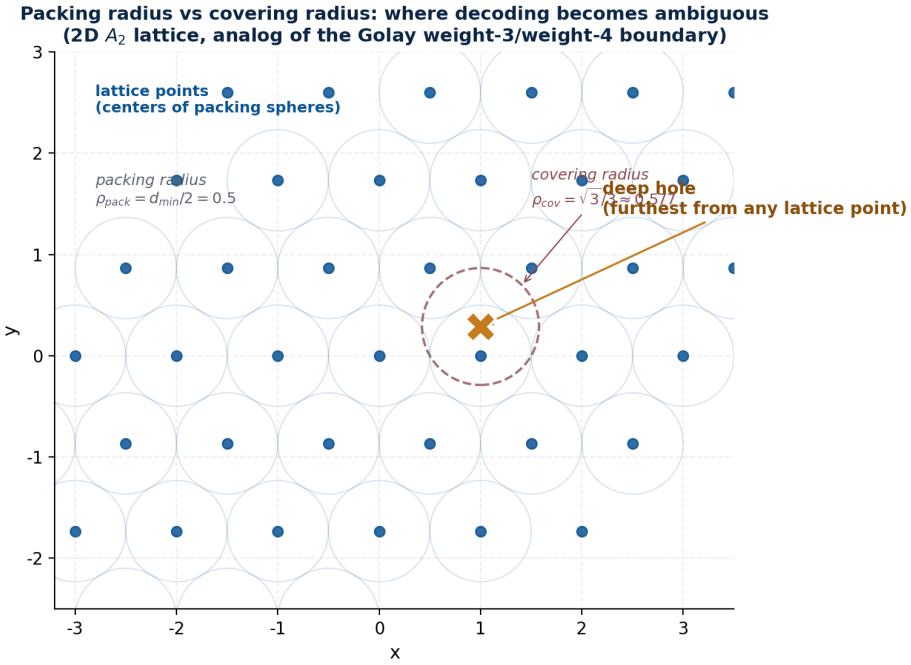

Figure 8. Packing radius vs. covering radius, illustrated on the 2D A_2 lattice (the hexagonal lattice, root system of $SU(3)$). The packing radius (half the minimum distance) defines the unique-decoding region; the covering radius (distance to the deep hole, the centre of the triangle) defines the failure boundary. The GLM's snap boundary at Hamming weight 4 is the binary analogue of the deep hole.

Sketch. The packing radius is half the minimum distance, $8/2 = 4$; the covering radius is computed from the coset-leader table as 4. The six equidistant codewords at distance 4 form the six tetrads of a sextet, and the system reports `ambiguous` with all six rather than picking one. The full proof is in `GolayBoundary.lean`. [snap_boundary_at_three *in Lean*] □

The snap boundary is the binary-code analogue of the deep hole. The GLM's substrate decoder reports `ambiguous` at Hamming weight 4 rather than picking one of the six equidistant codewords, which is the analogue of a lattice decoder reporting `deep hole` rather than picking an arbitrary nearest lattice point.

7.3 The 23 Niemeier Deep Holes

The Leech lattice is the unique even unimodular lattice in 24D with no roots, but it is not the only even unimodular lattice in 24D. There are 23 others, the Niemeier lattices, each with a distinct root system (A_1^{24}, A_2^{12}, A_3^8, A_4^6, A_6^4, A_8^3, A_{12}^2, D_4^6, D_6^4, D_8^3, D_{12}^2, $D_{16}E_8$, D_{24}, E_6^4, E_8^3, $A_5^4D_4$, $A_7^2D_5^2$, $A_9^2D_6$, $A_{11}D_7E_6$, $A_{15}D_9$, $A_{17}E_7$, A_{23}, $D_{16}^+E_8$). The 23 Niemeier lattices are in bijection with the 23 types of deep holes in the Leech lattice: each deep hole, when its nearest lattice points are collected and their convex hull examined, gives a Niemeier root system. This is the Conway–Parker theorem [2].

The GLM's deep-hole walk (Section 8) is a partial realisation of this correspondence: the 24-dimensional carrier, walked along the trajectory determined by the deterministic Delta–Sigma feedback, visits a sequence of Leech lattice points whose frequency distribution encodes the target. The generalisation of the deep-hole census over the 23 Niemeier lattices is named in the main paper as an open frontier.

8 Discussion: The Substrate is the Geometry

The two studies of this paper—the iteration test and the lattice survey—are two views of the same claim. The iteration test shows that floating-point arithmetic cannot natively represent rational trajectories over odd primes, and that the drift accumulates geometrically under expansive dynamics, reaching catastrophic magnitude (10^{19} at $p = 3$ with display6 handoffs) within 200 steps. The lattice survey shows that the substrate the GLM uses—the 24-coordinate Leech lattice, reached by the Construction A $\to$ B $\to$ C ladder from the binary Golay code—is one of seven canonical code-lattice pairings, each of which is a discrete-algebraic object (a code) lifted to a continuous-geometric object (a lattice) by a systematic congruence construction.

The two views meet at the snap boundary. The Hamming weight boundary (weight ≤ 3 correctable, weight 4 ambiguous, weight ≥ 5 failure) is the binary-code analogue of the lattice deep-hole boundary (within packing radius unique, at deep hole ambiguous, beyond covering radius failure). The GLM's substrate decoder reports the ambiguity rather than picking an arbitrary nearest codeword, which is the binary analogue of a lattice decoder reporting the deep hole rather than picking an arbitrary nearest lattice point.

8.1 The UBP Operationalised

The Universal Binary Principle's prohibition on floats is operationalised by the iteration test: floats drift, and the drift is untracked. The UBP's commitment to a 24-dimensional substrate is operationalised by the lattice survey: the substrate is one of seven canonical code-lattice pairings, each with its own geometry of decision boundaries (packing radius, covering radius, deep holes). The two commitments are not independent: the no-floats rule forces exact arithmetic, and exact arithmetic requires a substrate on which to compute; the substrate-native commitment requires a substrate, and the Leech lattice is the substrate because it is the unique even unimodular lattice in 24D with no roots, which means it is the unique 24D lattice whose geometry is not constrained by any root system, which means it is the unique 24D lattice that can carry arbitrary dimensional structure without bias.

8.2 The Dynamic Carrier: Potential, Not Storage

The main paper's Dynamic Carrier study (deterministic Delta–Sigma modulation on the weight-3/4 boundary) takes on a new light in the context of the lattice survey. The GLM does not store irrational numbers — it stores the *process* that converges to them. The Delta–Sigma modulator is a lattice decoder driven by an integrator: the integrator accumulates the error between the target and the last snap, the snap forces the carrier back to the nearest lattice point, and the trajectory of snaps encodes the target as a frequency distribution. No irrational is ever stored in a carrier; what is stored is the deterministic process whose time-average converges to the target at $O(1/N)$.

This is the sense in which the Dynamic Carrier is about *potential*, not storage. The carrier is always a finite lattice point (a rational integer vector in Λ_{24}). The potential to represent an irrational lies in the trajectory of repairs, not in any single state. The self-organised criticality at the weight-3/4 boundary (mean weight exactly 3.0 in the 24-D Golay modulator) is the deep-hole phenomenon: the carrier oscillates between weight 2 (within the packing radius, stable) and weight 4 (at the deep hole, critical), encoding the target as the frequency distribution of the two visits. The irrational is the limit of this process; it is never stored.

The generalisation to the other six canonical pairings is straightforward in principle: each code-lattice pair has its own packing radius, covering radius, and deep-hole structure, and each admits a Delta–Sigma modulator that uses the lattice's decoder as the snap. The convergence rate would depend on the covering radius and the dimension, with the Leech lattice's 196,560-

kissing geometry giving the densest snap space and therefore the finest resolution per step. The exploration of this generalisation is named as an open direction.

8.3 Open Questions

Three open questions arise from this study. First, the iteration test uses a specific parametric family of recurrences; a more general characterisation of when float drift is bounded (contractive dynamics) vs. explosive (expansive dynamics) would be valuable. Second, the Construction A $\rightarrow$ B $\rightarrow$ C ladder is presented for the binary Golay/Leech case; the analogous ladders for the other six pairings (parity/D_4, Hamming/E_8, ternary Golay/K_{12}, Reed-Muller/BW_{16}, QR/Q_{32}, Type II/P_{48n}) would complete the survey. Third, the deep-hole census over the 23 Niemeier lattices, which would generalise the GLM's snap boundary and the Conway–Parker theorem, is named in the main paper as an open frontier.

9 Conclusion

We have presented two studies that operationalise the Universal Binary Principle on which the GLM is built. The iteration test shows that floating-point arithmetic cannot natively represent rational trajectories over odd primes, and that the drift accumulates geometrically under expansive dynamics, reaching 10^{19} magnitude drift after 200 handoffs in the realistic display6 regime. The lattice survey shows that the GLM's 24-coordinate Leech substrate is one of seven canonical code-lattice pairings, reached from the binary Golay code by the Construction A $\rightarrow$ B $\rightarrow$ C ladder with kissing number growing 48 $\rightarrow$ 98,256 $\rightarrow$ 196,560.

The two studies together demonstrate that the UBP is not a methodological preference but a structural necessity. The no-floats rule is necessary because floats drift and the drift is untracked. The substrate-native commitment is necessary because exact arithmetic requires a substrate on which to compute, and the Leech lattice is the unique 24D substrate that can carry arbitrary dimensional structure without bias. The substrate is the geometry, and the geometry is the substrate.

References

[1] Euan R. A. Craig. *Geometric Language Machine: Repository Documentation.* 2026. https://github.com/DigitalEuan/GLM.

[2] John H. Conway and Neil J. A. Sloane. *Sphere Packings, Lattices and Groups*, volume 290 of *Grundlehren der mathematischen Wissenschaften.* Springer-Verlag, New York, 3 edition, 1999.

[3] Henry Cohn, Abhinav Kumar, Stephen Miller, Danylo Radchenko, and Maryna Viazovska. The sphere packing problem in dimension 24. *Annals of Mathematics*, 185(3):1017–1033, 2017.